

Rethinking LLM-aided RTL Code Optimization Via Timing Logic Metamorphosis

ZHIHAO XU, SouthEast University, China

BIXIN LI*, SouthEast University, China

RAN YAN, SouthEast University, China

LULU WANG, SouthEast University, China

YONGQIANG TIAN, Monash University, Australia

Register Transfer Level (RTL) code optimization is critical for meeting performance and power budgets in Field Programmable Gate Array (FPGA) design. Traditional approaches depend on expert knowledge or heuristics, which are time consuming and error prone. Therefore, recent research has explored using Large Language Models (LLMs) for RTL code optimization and providing initial evidence of their potential. However, the capabilities and limitations of LLMs in RTL code optimization have not been systematically assessed, especially for RTL with complex timing logic. Moreover, these studies cannot distinguish whether improvements come from genuine reasoning or merely reproducing memorized patterns from training data. To address these problems, we present a metamorphic testing based evaluation method and a companion benchmark to measure the effectiveness of LLM-aided RTL code optimization. Our key idea is that optimization effectiveness should be consistent across RTL descriptions that are semantically equivalent. We first build a benchmark suite covering four domains (logic operations, datapaths, timing control flow, and clock domains). We then apply domain specific transformations to produce semantically equivalent RTL code with different code forms. We assess each method by applying it to both the original and the transformed code, synthesizing them with the same flow, and comparing the synthesis results. Through extensive experiments, our results show that LLM-aided methods effectively optimize logic operations and data paths, achieving lower or stable wire counts and area with delays near baseline. For timing control flow and clock domains the gains are limited and structural ratios often rise. Larger code size also increases the chance of incorrect rewrites, especially in timing and cross domain designs. We further explore the application of prompt engineering include few shot examples and Chain of Thought (CoT) to improve RTL code optimization. Based on these results we discuss practical directions and give concrete suggestions for using LLMs in RTL optimization.

CCS Concepts: • **Hardware** → **Logic synthesis**; **Circuit optimization**.

Additional Key Words and Phrases: Empirical study, LLMs, Optimization, RTL code

1 Introduction

Optimizing Register Transfer Level (RTL) code is a critical step in the early stages of Field Programmable Gate Array (FPGA) design [24, 47, 52, 62]. This process involves iteratively rewriting RTL code snippets into optimized

*Corresponding author.

Authors' Contact Information: Zhihao Xu, Hosea.xu@seu.edu.cn, SouthEast University, China; Bixin Li, bx.li@seu.edu.cn, SouthEast University, China; Ran Yan, 220242339@seu.edu.cn, SouthEast University, China; Lulu Wang, wanglulu@seu.edu.cn, SouthEast University, China; Yongqiang Tian, yongqiang.tian@monash.edu, Monash University, Australia.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1936-7414/2026/9-ART

<https://doi.org/10.1145/3847661>

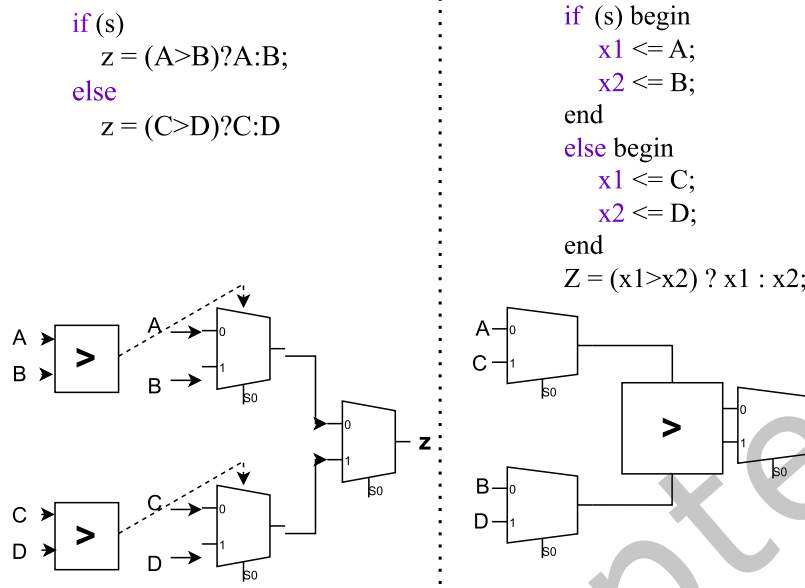


Fig. 1. Example of LLM-aided MUX optimization of RTL code

RTL code based on optimization patterns or synthesis feedback. Traditionally, this task has relied on the expertise of seasoned engineers [72].

However, the growing complexity of design patterns poses challenges for manual optimization. While conventional automated methods provide alternatives, they suffer from limited optimization scope and often produce suboptimal results for complex RTL code [64]. These limitations manifest as longer critical path delays or lower maximum frequencies when dealing with complex RTL designs.

To address these limitations, recent research has explored the application of Large Language Models (LLMs) for RTL optimization [72]. LLMs can comprehend the high-level semantic intent of RTL designs to explore a broader design space, thereby solving the problem of suboptimal results produced by the more rigid, heuristic-based approaches of conventional tools. For example, Figure 1 shows a classic case of LLM-aided multiplexer (MUX) optimization. In this case, the LLM-aided approach moves conditional comparison operations before MUX selection. By avoiding redundant execution of arithmetic and comparison logic, this transformation reduces hardware resource usage and dynamic power consumption. In contrast, conventional optimization tools such as Yosys [69] typically fail to identify this optimization. They follow the structure of code literally, first creating the multiplexer and then the shared comparison logic.

LLM-based approaches have demonstrated distinct advantages over conventional methods for certain RTL optimization tasks. However, the true capabilities and limitations of these approaches remain uncertain due to a lack of comprehensive evaluation. This uncertainty stems from two critical shortcomings in current benchmarking practices. First, existing benchmarks are heavily skewed towards combinational logic optimizations, such as datapath restructuring and MUX simplification. They largely neglect optimizations involving complex timing and sequential logic. This limitation is significant, as timing characteristics ultimately determine a circuit's real-world performance, power efficiency, and reliability. Second, existing benchmark suites are often small and may lack statistical power. More critically, they risk data contamination because many canonical examples may have appeared in the LLMs' pre-training data. This contamination complicates assessment of the true generalization

ability of a model, making it difficult to determine whether the model is optimizing novel code or merely recalling memorized solutions.

To overcome these evaluation challenges, we propose a two-part solution. To address the limited size and narrow scope of existing suites, we first constructed a new benchmark. We gathered RTL code from various benchmarks and open-source projects. With guidance from experienced RTL developers, we curated this collection to ensure it includes a diverse set of designs, with a specific focus on code containing complex timing and sequential logic. We categorize RTL optimization into four key domains: *logic operation optimization*, *datapath optimization*, *timing control flow optimization*, and *clock domain optimization*. These four categories reflect guidelines from FPGA vendors, EDA tool documentation, and industry best practices [2, 6, 9, 16, 51, 54, 59, 60, 74], providing a structured basis for assessing optimization effectiveness under varying levels of RTL complexity.

To properly assess the generalization ability of models and mitigate data contamination risks, we introduce a method based on metamorphic testing. For each domain, we design a metamorphic strategy to generate semantically equivalent but structurally different RTL code. We refer to these generated code snippets as RTL mutants.

Our key insight is that the effectiveness of RTL optimization should remain consistent under semantically equivalent but structurally more complex transformations. This expectation is grounded in industrial practice, where engineers routinely introduce structural variations for reuse purposes without changing functional semantics. Existing synthesis tools consistently optimize across such variations to achieve similar timing and area results. Accordingly, practical LLM-aided optimization methods should also be robust to these structural changes.

We utilized our new benchmark and metamorphic testing framework to conduct a comprehensive evaluation. In our experiments, we applied both LLM-aided and traditional compiler-based optimization methods to the original RTL designs and their generated mutants. We then systematically compared the outcomes by analyzing key circuit metrics after synthesis, including timing performance, area utilization, and power consumption.

Our results show that across the four domains LLM aided methods can improve logic operations and data paths. Wire counts and area decrease or stay close to the Yosys baseline and delays remain near the baseline. These gains persist on semantically equivalent mutants, which indicates that the improvements are not limited to a single code form. The observed reductions are consistent with simplification of conditional selection logic and removal of redundant terms while cycle boundaries are preserved. But for timing control flow and clock domains, improvements are limited. On mutants we observe higher wire and cell ratios and only small changes in area, delay, and power. These outcomes align with added states, longer selection chains, additional registers, and interface logic introduced during rewriting. The chance of incorrect rewrites increases with code size and the rise is most pronounced for timing control flow and clock domains. A segmentation based pipeline such as RTLRewriter lowers these error rates by constraining the edit scope and supplying targeted examples, but it does not remove the limits we observe on area, delay, and power. Prompt design helps mainly on logic operations and data paths. Few shot examples and chain of thought bring small but consistent gains by anchoring common refactoring templates and encouraging step by step planning, while their effect on timing control flow and clock domains is modest because they do not change cycle boundaries or synchronizer placement. Finally we also discuss practical routes for timing related RTL optimization and point out open problems.

Our contributions are summarized as follows:

- We introduce a metamorphic testing method to evaluate the effectiveness of LLM-aided RTL optimization in handling complex RTL code, particularly those with intricate timing logic.
- To the best of our knowledge, this is the first systematic study that uses metamorphic testing to evaluate the robustness of LLM-aided optimization on existing RTL code, especially for timing-complex designs.

- We provide a new benchmark suite for evaluating the effectiveness of RTL optimization methods on timing-critical RTL designs¹.

2 Background and Motivation

In this section, we first introduce the background of RTL code and its timing semantics. We then present the motivation for our work with a simple example.

2.1 RTL Code

RTL descriptions are typically written in hardware description languages (HDLs) such as Verilog or VHDL. The core distinction between RTL code and conventional software code lies in RTL's inherently concurrent and synchronous temporal semantics [70, 71].

In software programming languages (e.g., C, Java, or Python), instructions execute sequentially, and variables are updated instantaneously within the thread of execution. By contrast, RTL code expresses system behavior as a set of concurrent processes that evolve synchronously in response to a global timing reference, typically a clock signal. This fundamental difference means that RTL optimization must account for not only functional correctness but also timing constraints, signal dependencies, and concurrent execution patterns.

For example, as shown in Listing 1, the counter value is updated only on the rising edge of the clock. When the reset signal is asserted, the counter is synchronously reset to zero. When the counter reaches 15 (4'b1111), it wraps around to zero on the next clock cycle; otherwise, it increments by one. This example demonstrates that RTL optimization requires explicit consideration of timing semantics. Both what happens and when it happens are critical for correct circuit behavior.

```

module counter(
  input wire clk,
  input wire reset,
  output reg [3:0] count);
always @(posedge clk or posedge reset) begin
  if (reset)
    count <= 4'b0000;
  else if (count == 4'b1111)
    count <= 4'b0000;
  else
    count <= count + 1'b1;
end
endmodule

```

Listing 1. Example of a simple counter in RTL

The temporal nature of RTL code introduces unique optimization challenges. Consider a simple datapath optimization, while conventional code optimization might freely reorder independent operations for better performance, RTL optimization must preserve cycle-accurate behavior. A transformation that appears functionally equivalent may introduce timing violations, create race conditions, or alter the circuit's critical path. These subtleties make RTL optimization fundamentally more complex than conventional code optimization.

2.2 Illustrative Example

Existing LLM-aided RTL optimization methods rely on the semantic understanding capabilities of LLMs, often combined with auxiliary search mechanisms [72]. These approaches assume that LLMs can comprehend both the functional behavior and temporal characteristics of RTL designs. However, the ability of LLMs to accurately

¹<https://github.com/zhihaoxu1325/RTLopt>

capture timing semantics in RTL code has not been systematically validated [34, 55, 57]. This gap in validation is particularly concerning given the critical role that timing plays in hardware design.

To illustrate this limitation, we present an empirical study inspired by mutation testing principles. Given a program snippet P , a language model encodes it into a continuous vector representation $E(P) \in \mathbb{R}^d$, where d is the embedding dimension. A fundamental assumption in code understanding is that semantically similar programs yield embeddings with smaller distances in the embedding space [27, 46]. If this assumption holds for RTL code, then semantically equivalent RTL snippets should cluster together, while semantically different ones should be separated.

We generated numerous RTL test cases using recent code generators [71] and applied two categories of carefully designed mutations:

- **Semantics-preserving mutations:** These transformations maintain the exact functional and temporal behavior while altering the structural representation. Examples include restructuring nested conditionals into flattened forms, injecting redundant but neutral logic operations, rewriting always blocks with equivalent control flow patterns, and refactoring between blocking and non-blocking assignments where semantically appropriate. These mutations test whether LLMs can recognize functional equivalence despite structural differences.
- **Semantic-breaking mutations:** These transformations alter either the functional behavior or timing characteristics of the circuit. Examples include inverting logical conditions, introducing temporal inconsistencies such as incorrect clock edge sensitivities, modifying thresholds that control state transitions, altering signal dependencies that affect timing paths, and introducing incorrect variable updates that change the circuit's functionality. These mutations test whether LLMs can distinguish between correct and incorrect RTL implementations.

By comparing embedding distances between the original and mutated RTL snippets, we assess how well LLMs capture both structural and temporal semantics. In an ideal scenario, all semantics-preserving mutations should cluster near the original code, while semantic-breaking mutations should be clearly separated.

Figure 2 shows t-SNE projections of embeddings from *CodeLlama-7B* [49], a state-of-the-art code understanding model. The blue point represents the original program, green points denote semantics-preserving mutations, and red points indicate semantic-breaking mutations. The results reveal concerning patterns in the model's understanding of RTL semantics.

Most notably, we observe that the MR-3 group, which consists of semantically equivalent transformations involving timing restructuring, appears farther from the original code than several semantic-breaking mutants. This counterintuitive result suggests that the model fails to recognize functional equivalence when timing-related transformations are involved. Furthermore, several semantic-breaking mutations that alter critical timing behavior cluster close to the original code, indicating that the model cannot reliably distinguish between correct and incorrect timing implementations. These findings have significant implications for LLM-aided RTL optimization. If LLMs cannot accurately differentiate between semantically equivalent and non-equivalent RTL code, particularly when timing semantics are involved, their optimization suggestions may introduce subtle bugs or fail to identify valid optimization opportunities. The misalignment between embedding distances and semantic similarity suggests that current LLMs may lack the specialized understanding required for timing-aware RTL optimization.

We further repeat the same embedding analysis on C programs. We construct semantics-preserving C variants through equivalent Boolean rewriting, temporary variable refactoring, and redundant branch insertion, and construct semantic-breaking C mutants through condition inversion, boundary modification, and operator replacement. Compared with the RTL results in Figure 2, the C programs show a clearer separation between semantics-preserving variants and semantic-breaking mutants in the embedding space. In particular, semantics-preserving C variants remain closer to the original program, while semantic-breaking mutants are generally

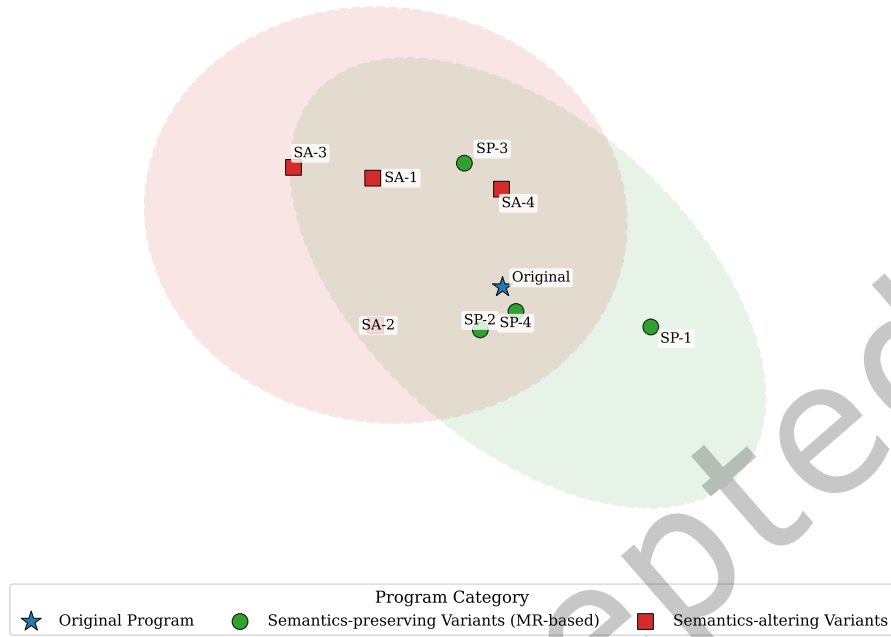


Fig. 2. Embedding results of original and mutated code on CodeLlama

mapped farther away. By contrast, RTL variants exhibit weaker separation, especially for timing-related transformations, where cycle equivalent variants can be mapped farther from the original design than some semantic breaking mutants.

This observation motivates a deeper investigation into whether LLMs can reliably assist engineers in optimizing RTL designs that contain timing-critical logic. While LLMs have shown promise in optimizing combinational logic and simple datapath structures, their ability to handle the full complexity of RTL optimization, especially for designs with intricate timing requirements, remains an open question that requires systematic evaluation.

3 Methodology

In this section, we present our methodology for evaluating the effectiveness of LLM-aided RTL code optimization methods in handling complex RTL designs, particularly those with intricate timing logic. As shown in Figure 3, our approach consists of three stages: *Test Case Classification*, *RTL Code Metamorphosis*, and *Quality Evaluation*. Specifically, we first collect existing benchmark RTL datasets and classify them into four key areas of RTL optimization. Next, in the *Metamorphosis* stage, we apply transformation strategies to the original benchmark test cases to generate new, functionally equivalent mutants (referred to as *mutants*). Finally, in the *Quality Evaluation* stage, we assess the effectiveness of LLM-aided RTL optimization methods by comparing results on the original RTL code and its mutants.

Our methodology is inspired by existing code symmetry evaluation techniques [25, 58]. We adopt a similar idea of metamorphosis to systematically assess the effectiveness of LLM-aided RTL code optimization when facing structurally complex yet semantically equivalent designs.

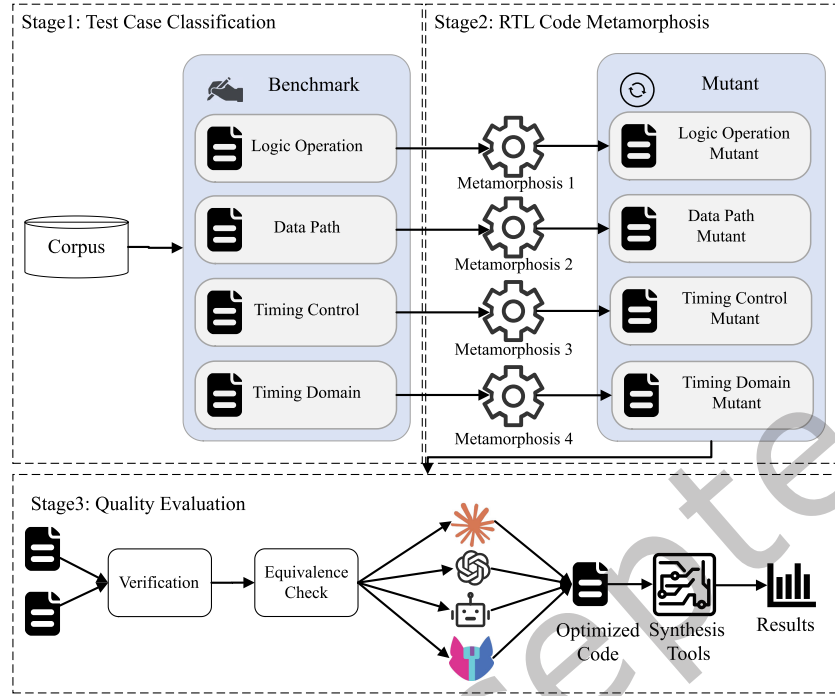


Fig. 3. Workflow of our Method to evaluate LLM-aided RTL Optimization Method

3.1 Test Case Classification

To construct a benchmark for RTL optimization, we first collected RTL code from existing benchmarks [35, 38], open-source repositories on GitHub, and OpenCores [43]. We then invited domain experts to classify the collected RTL designs into four key areas of optimization: *logic operation optimization*, *data path optimization*, *timing control flow optimization*, and *clock domain optimization*. Through multiple rounds of screening and evaluation, we ensured that the RTL code in each category is functionally non-overlapping and representative. Thus, our benchmark consists of four distinct parts, each containing RTL code snippets representative of the corresponding optimization domain. To clarify the role of our taxonomy, the four benchmark categories do not constitute a complete or strictly disjoint ontology of real-world RTL designs. Instead, they are analysis-oriented categories used to capture the dominant optimization challenge of each benchmark case. When a design exhibits multiple characteristics, we assign it to the category that best reflects its primary structural feature and the main source of optimization difficulty. We further summarize the classification criterion in Table 1, which defines each category in terms of its representative RTL characteristics and primary optimization challenge.

3.2 RTL Code Metamorphosis

In this stage, we apply domain-specific metamorphosis strategies to the four key areas of RTL optimization. In this work, metamorphic testing is not used to simplify the optimization task for LLMs. Instead, it is used to construct semantics preserving but structurally perturbed RTL variants from the same original design. The purpose is to test whether an LLM based optimizer can maintain consistent optimization behavior when the surface form of the RTL changes while the underlying design semantics remain unchanged.

Table 1. Definition of the four benchmark categories.

Category	Representative RTL characteristics	Primary optimization challenge
Logic Operation	Boolean expressions, predicates, logic cones, and condition simplification	Local combinational rewriting and Boolean simplification
Data Path	Arithmetic operators, mux chains, accumulators, data-movement structures, and wide buses	Arithmetic and data-movement restructuring under regular single-clock execution
Timing Control Flow	FSMs, counters, staged control, phase transitions, and latency-sensitive control logic	Cycle-level scheduling, state-transition restructuring, and timing-sensitive control preservation
Clock Domain	Synchronizers, asynchronous FIFOs, CDC wrappers, cross-clock handshakes, and reset-domain interactions	Correctness-preserving optimization under multiple clocks or cross-domain interactions

This methodology enhances the evaluation in three ways. First, it provides a systematic way to generate semantically controlled benchmark variants without changing the intended functionality of the original design. Second, it enables robustness analysis by allowing us to compare optimization results on the original RTL program and on its equivalent metamorphic variants under the same evaluation setting. Third, it helps distinguish true semantic understanding from superficial pattern dependence: if two RTL programs are semantically equivalent but lead to substantially different optimization outcomes, then the optimizer is likely sensitive to source level representation rather than the underlying design behavior.

For this reason, the examples in Section 3.2 intentionally increase structural complexity instead of simplifying it. This is a deliberate design choice. The added complexity allows us to stress test whether the optimizer can preserve correctness and optimization consistency under semantics preserving perturbations. These strategies are introduced below.

3.2.1 Logic Operation Optimization. For logic operation optimization, we employ a metamorphosis strategy based on Boolean algebra laws. The strategy applies a series of algebraic transformations to restructure RTL expressions while preserving their semantics.

```

module logic_transformed(
  input a, b, c,
  output y
);
- assign y = (a & b) | (a & c);
+ wire term1 = ~(~a | ~b);
+ wire term2 = ~(~a | ~c);
+ wire term3 = ~(~term1 & ~term2);
+ wire term4 = ~(~(a & b & c));
+ assign y = ~(~term3 & ~term4);
endmodule

```

Listing 2. Example of metamorphosis strategy in logic operation optimization

For example, let $E = (a \wedge b) \vee (a \wedge c)$ denote the original expression. We decompose E into its sub-expressions $(a \wedge b)$ and $(a \wedge c)$, then apply De Morgan's laws to obtain the NOR equivalents $\neg(\neg a \vee \neg b)$ and $\neg(\neg a \vee \neg c)$. These are recombined into a single nested expression, converting the disjunction into a conjunction structure. Thus, the original expression E is transformed into a more complex form, as shown in Equation 1.

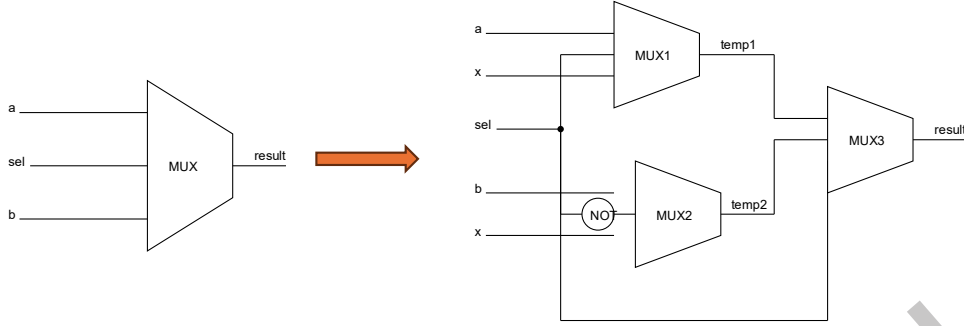


Fig. 4. Examples of metamorphosis strategies in data path optimization

$$E \Rightarrow \neg[\neg(\neg(\neg a \vee \neg b)) \wedge \neg(\neg(\neg a \vee \neg c))] \quad (1)$$

Listing 2 illustrates this transformation process. Starting from a simple Boolean expression, we iteratively introduce redundant terms to increase syntactic complexity while maintaining logical equivalence. Each step replaces basic operations with more intricate forms and adds redundant sub-expressions, resulting in a functionally equivalent yet more complex Boolean expression.

3.2.2 Data Path Optimization. For data path optimization, we propose a cascade metamorphosis strategy. This strategy increases data-flow complexity and diversifies multiplexer structures by decomposing direct assignments and selection logic into multi-stage, cross-referenced configurations.

Specifically, we first identify key signals in the RTL code and construct condition expressions that are always true. We then insert conditional branches using these expressions as predicates. The original functional code is placed in the true branch, while the else branch contains randomly generated logic that is never executed. This modification increases static complexity without changing semantics.

Second, for multiplexer structures, we transform standard single-stage multiplexers into nested or cascaded forms. As shown in Figure 4, the left diagram depicts a basic two-to-one multiplexer. Using the metamorphosis strategy, we refactor it into the structure on the right by introducing an intermediate variable X . The circuit first selects between two inputs and X , and then produces the final output. This additional layer increases multiplexer complexity while preserving original semantics.

3.2.3 Timing Control Flow Optimization. For timing control flow optimization, we propose a state-machine-based metamorphic strategy to evaluate whether LLM-aided optimizers can handle complex sequential logic. This strategy increases FSM structural complexity while strictly preserving cycle-accurate timing behavior. It replaces direct state transitions with chained sequences of pass-through states and subdivides multi-operation states into multiple sub-states. For example, a direct transition $S_1 \rightarrow S_2$, where S_1 originally lasts N clock cycles, can be expanded to $S_1 \rightarrow S_{1A} \rightarrow S_{1B} \rightarrow S_{1C} \rightarrow S_2$. To maintain timing equivalence, the duration of S_1 is reduced from N to $N - K$ cycles, where K is the number of inserted pass-through states, so that the total cycle count from entering S_1 to reaching S_2 remains exactly N . Each pass-through state occupies exactly one clock cycle and produces the same output as S_1 , ensuring that the externally observable signal remains unchanged at every cycle. Similarly, a state performing updates to both A and B can be split into two sub-states, one updating A and the other updating B , with their combined duration equal to that of the original state.

As illustrated in Figure 5, we applied this strategy to a traffic signal controller. In the original code, state HWY_GREEN transitions directly to HWY_YELLOW. In the mutant version, three intermediate states were

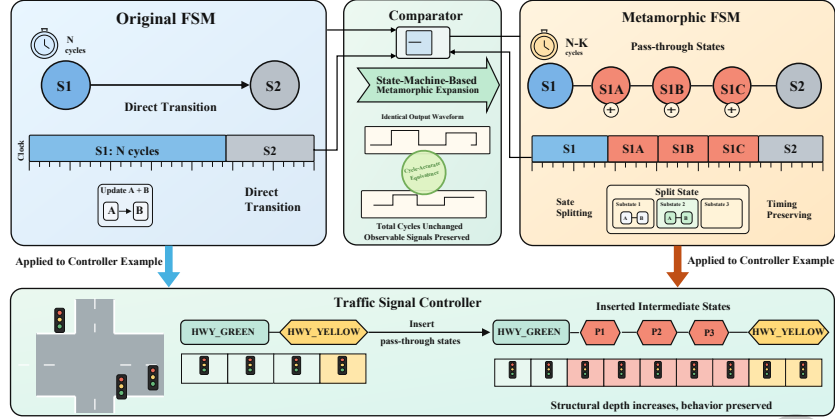


Fig. 5. Timing control flow metamorphosis Example

inserted, increasing state complexity without altering behavior. Similarly, a single state lasting six cycles was subdivided into three semantically equivalent sub-states, preserving overall timing but increasing structural depth.

3.2.4 Clock Domain Optimization. For clock domain optimization, we propose a cross-domain metamorphosis strategy. This strategy partitions a single clock domain into multiple domains with identical or frequency-multiplied clocks, inserting synchronizers and additional logic between them. Although functionality remains unchanged, the design becomes more heterogeneous and structurally complex.

We also replace standard synchronizers with more intricate protocols. These modifications preserve semantics but introduce additional synchronization complexity. As shown in Listing 3, the original single-clock logic transferring data from regA to regB is refactored into a two-clock structure with synchronizer registers, while maintaining functional equivalence.

```

- always @(posedge clk) begin
-   regA <= ...;
-   regB <= comb(regA);
- end
+ always @(posedge clk1) begin
+   regA <= ...;
+   sync_reg1 <= comb1(regA);
+ end
+ always @(posedge clk2) begin
+   sync_reg2 <= sync_reg1;
+   regB <= comb2(sync_reg2);
+ end

```

Listing 3. Example of metamorphosis strategy in clock domain optimization

3.3 Quality Evaluation

After applying the metamorphosis strategies, we obtain a new benchmark of RTL mutants. We then apply different optimization methods to both the original and mutant RTL code. To ensure functional equivalence, we conduct formal verification with ABC [69] and simulation with Icarus Verilog [1]. Subsequently, we synthesize both

the original and mutant RTL using a standard compiler to obtain gate-level descriptions. Theoretically, since metamorphosis preserves semantics, both versions should yield equivalent optimization results. Therefore, by comparing synthesis outcomes on the original and mutant code, we can systematically evaluate the effectiveness and robustness of RTL optimization methods.

4 Evaluation

In this section, four experiments are conducted to evaluate the effectiveness of *LLM-aided* RTL code optimization methods in handling complex timing logic. Specifically, our evaluation aims to answer the following Research Questions (RQs).

- RQ1: In which aspects are LLM-aided RTL optimization methods effective?
- RQ2: In which aspects do LLM-aided RTL optimization methods show limited effectiveness?
- RQ3: In which aspects do LLM-aided RTL optimization methods produce failure results?
- RQ4: Can existing prompt engineering methods improve the RTL code optimization effectiveness of LLM-aided RTL optimization methods?

4.1 Baseline

We selected RTLRewriter [72] as one of our baselines. RTLRewriter optimizes RTL code using LLMs by partitioning the code into smaller segments and using retrieval-augmented generation to strengthen the effectiveness of LLMs in optimization. It is the latest and one of the most effective *LLM-aided* methods for RTL code optimization. We also selected GPT-4 and Claude-3.7-Sonnet as baseline models for comparison in our experiments, since they are state-of-the-art large models for code-related tasks. The baseline method [72] also uses these two LLMs as primary comparisons. Furthermore, we included Yosys, the most widely used open-source compiler, consistent with RTLRewriter.

4.2 Benchmark

To construct a benchmark for RTL optimization, we first collected RTL code from existing benchmarks [35, 38], open-source repositories on GitHub, and OpenCores [43]. Our benchmark includes basic patterns, data path, memory, MUX, FSM, and control logic. In total, our benchmark includes 54 cases related to logic operations, 27 related to data paths, 40 related to timing control flow, and 32 related to clock domains.

4.3 Implementation

We implemented our metamorphosis strategies using Python and Verilog. The evaluation ran on a computer with an Intel Core i9 CPU @ 2.10 GHz, 128 GB RAM, and an NVIDIA A6000 GPU (CUDA 12.0). We evaluated five metrics commonly used in RTL optimization and synthesis, wires, cells, area, delay, and power. They are defined as follows:

- **Wires:** interconnections in an RTL design. A lower number generally indicates a more efficient implementation.
- **Cells:** logical elements utilized in the design; a higher count signifies greater logical complexity.
- **Area:** physical space occupied by the design, including the number of logic gates, flip-flops, and interconnections required to implement it. Smaller area indicates a more compact implementation.
- **Delay:** time for a signal to propagate through the design. Lower delay indicates faster performance.
- **Power:** power consumption of the design. Lower power indicates higher energy efficiency.

4.4 Prompt Design

Our prompt design is informed by the baseline approach [72], which employs a large language model to perform RTL optimization under natural-language instructions, without extensive prompt engineering or task-specific demonstrations. Following this paradigm, we adopt a fixed zero-shot prompt across all models and experiments to eliminate prompt variability as a confounding factor. Specifically, our prompt is as follows.

[Prompt Template]

You are an expert hardware engineer optimizing RTL code for performance and area.
Below is an RTL module. Please optimize it while keeping its functionality unchanged.

[RTL Snippet]

```
module foo (...);
...
endmodule
```

4.5 Metric Normalization

To assess the effectiveness of *LLM-aided* RTL optimization, we require a reference implementation that serves as a functional and structural baseline. We use the output of the industry-standard toolchain Yosys as the de facto ground truth. Specifically, for each input RTL design, we synthesize it using Yosys with standard optimization passes enabled (e.g., opt, techmap, abc) to obtain a canonicalized, optimized version. This synthesized result is treated as the reference against which we compare the output of *LLM-aided* optimization methods. Since Yosys is a mature, widely adopted open-source synthesis tool that applies deterministic and semantics-preserving optimizations, it provides a stable and reproducible target for evaluation. We use the Yosys result on the original RTL program of each benchmark instance as the normalization anchor. For this reason, Yosys_{org} is always 1.00 by the experimental setting, and Yosys_{mut} indicates how the optimization results of the mutant RTL code under the same Yosys optimization flow. We report all quantitative metrics (e.g., wire count, cell count, area, delay, power) as **normalized ratios** relative to **Yosys**. For a given metric M and RTL instance d , the normalized ratio under a method $\mathcal{A}$ is

$$\text{Ratio}_{\mathcal{A}}^{(d)} = \frac{M_{\mathcal{A}}^{(d)}}{M_{\text{Yosys}}^{(d)}}.$$

We then compare the **mean normalized ratio** of each method across designs for each key area. Normalization allows fair comparison across different RTL designs by evaluating relative performance change from the baseline.

4.6 Answer to RQ1

As shown in Table 2 and Table 3, LLM aided methods outperform the compiler baseline on logic operations and data paths. On structural metrics, LLM-aided methods perform better than the compiler baseline on both original RTL and mutated RTL. For logic operations, RTLRewriter and Claude keep wire and cell ratios at the baseline level on both code forms. GPT-4 shows a small increase in wires on mutated RTL while cells remain unchanged. Yosys increases notably on mutated RTL. For data paths, all LLM-aided methods and RTLRewriter reduce or maintain wire ratios and keep cell ratios at the baseline level, whereas Yosys increases on mutated RTL.

On PPA metrics, we map designs to the same standard-cell library and report ratios normalized to Yosys on the original RTL. For logic operations and data paths, the preliminary PPA results are consistent with the structural findings. Area is comparable to the baseline when cell counts remain unchanged and shows slight reductions when structural simplification affects selection networks. Critical-path delay generally remains close to the

baseline since the transformations preserve cycle boundaries. Power changes follow the direction of switching activity implied by the structural patterns and are small in magnitude.

Table 2. Structural results across four categories.

Method	Logic Operation		Data Path		Timing Control Flow		Clock Domain	
	Wires	Cells	Wires	Cells	Wires	Cells	Wires	Cells
RTLRewriter_org	1.00	1.00	0.84 ↓	1.00	0.59 ↓	0.38 ↓	1.58 ↑	0.11 ↓
RTLRewriter_mut	1.00	1.00	0.84 ↓	1.00	4.85 ↑	3.67 ↑	2.96 ↑	0.44 ↓
GPT_org	1.00	1.00	0.84 ↓	1.00	1.93 ↑	1.75 ↑	1.67 ↑	0.17 ↓
GPT_mut	1.17 ↑	1.00	0.89 ↓	1.00	1.26 ↑	0.12 ↓	3.25 ↑	0.72 ↓
Claude_org	1.00	1.00	0.84 ↓	1.00	1.00	1.00	1.67 ↑	0.22 ↓
Claude_mut	1.00	1.00	0.89 ↓	1.00	0.74 ↓	0.12 ↓	2.96 ↑	0.44 ↓
Yosys_org	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Yosys_mut	4.67 ↑	3.25 ↑	1.58 ↑	1.80 ↑	6.74 ↑	7.33 ↑	2.17 ↑	2.50 ↑

Wires and Cells with ↓ indicate improvements (<1), ↑ indicate regressions (>1).

These results indicate that LLM-aided optimization is effective for combinational refactoring and for restructuring multiplexer based data paths under our evaluation protocol. A plausible explanation is that the models can use semantic cues available in the code to remove redundant terms and to simplify nested conditionals or cascaded multiplexers without altering functional or temporal semantics. The compiler baseline adheres more closely to the surface structure and does not consistently collapse deep selection networks introduced by code transformations, which leads to increases on mutated RTL. Furthermore, mild regressions can occur when an LLM materializes intermediate computations or retains non executing branches, which increases nets and switching without clear benefits in cells. For example, GPT-4 sometimes materializes intermediate computations as separate continuous assignments and then selects among them in the always block. Listing 4 shows such a case.

```

wire [7:0] sum_result = a + b;
wire [7:0] diff_result = a - b;
wire [7:0] and_result = a & b;
wire [7:0] or_result = a | b;

always @(+) begin
  case (op_code)
+   2'b00: result = sum_result;
+   2'b01: result = diff_result;
+   2'b10: result = and_result;
+   2'b11: result = or_result;
    default: result = 8'b0;
  endcase
end

```

Listing 4. RTL code optimization example of GPT-4

This pattern preserves functionality but increases the number of nets synthesized, which explains the small wire increase on mutated RTL while cell counts remain unchanged.

Overall. With all metrics normalized to Yosys on the original RTL under an identical synthesis and timing setup, LLM-aided methods are effective for logic operations and data paths. They achieve lower or stable wire and cell ratios compared with the compiler baseline and the measured PPA is consistent with these structural outcomes.

Table 3. PPA results across four categories.

Method	Logic Operation			Data Path			Timing Control Flow			Clock Domain		
	Delay	Area	Power	Delay	Area	Power	Delay	Area	Power	Delay	Area	Power
RTLRewriter_org	1.00	1.00	0.99 ↓	1.00	0.98 ↓	0.97 ↓	1.00	0.61 ↓	1.00	1.01 ↑	1.50 ↑	0.62 ↓
RTLRewriter_mut	1.00	1.00	0.99 ↓	1.00	0.98 ↓	0.97 ↓	1.01 ↑	1.02	1.00	1.09 ↑	1.64 ↑	0.77 ↓
GPT_org	1.00	1.00	1.00	1.00	0.98 ↓	0.98 ↓	0.91 ↓	1.78 ↑	1.00	1.00	0.98 ↓	1.00
GPT_mut	1.00	1.00	1.02 ↑	1.00	0.99 ↓	0.99 ↓	0.86 ↓	7.93 ↑	3.00 ↑	1.06 ↑	1.02	1.00
Claude_org	1.00	1.00	0.99 ↓	1.00	0.98 ↓	0.98 ↓	1.00	1.00	1.00	0.96 ↓	0.96 ↓	1.00
Claude_mut	1.00	1.00	0.99 ↓	1.00	0.99 ↓	0.99 ↓	1.00	0.55 ↓	1.00	1.03 ↑	1.02 ↑	1.00
Yosys_org	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Yosys_mut	1.02 ↑	3.10 ↑	1.30 ↑	1.03 ↑	1.60 ↑	1.10 ↑	1.01 ↑	2.93 ↑	1.00	1.07 ↑	1.04 ↑	1.00

4.7 Answer to RQ2

As shown in Table 2 and Table 3, LLM aided methods show limited improvements on timing control flow and clock domains. For timing control flow, structural ratios on mutated RTL increase for several methods. RTLRewriter shows increases in wires and cells on mutated RTL (4.85× and 3.67×), while its delay and area essentially match the baseline (1.01× and 1.02×). GPT_mut reduces delay to 0.86× but raises area and power substantially (7.93× and 3.00×). Claude_mut keeps delay at 1.00× and reduces area to 0.55×. Yosys_mut increases delay and area slightly (1.01× and 2.93×). For clock domain, increases in structural ratios on mutated RTL are observed across methods. Claude_mut yields delay and area close to baseline (1.03× and 1.02×). RTLRewriter_mut increases delay and area (1.09× and 1.64×) but reduces power to 0.77×. GPT_mut increases delay and area slightly (1.06× and 1.02×). Yosys_mut increases delay and area slightly as well (1.07× and 1.04×).

These results indicate that LLM-aided optimization is limited on designs that emphasize temporal behavior. For timing control flow, RTLRewriter partitions the RTL and optimizes submodules independently. This loses the global state transition graph and limits the removal of redundant states or the merging of equivalent transitions introduced by the mutated RTL. As a result, selection chains persist, wires and cells increase on mutated RTL, and delay remains close to the baseline because no state re-encoding or path shortening is applied. GPT-4 often rewrites an expanded transition into a more decomposed combinational structure with additional intermediate signals and logic cells. This can shorten the critical-path delay, but it may also increase the number of cells, nets, and dynamic power. For clock domain settings, RTLRewriter aligns partition boundaries with domain interfaces and applies local gating or buffering without global scheduling across domains. This raises area and can increase delay, while power may decrease when activity inside partitions is reduced. GPT-4 and Claude sometimes move selection logic across domain interfaces or widen synchronizer fan-out. This adds interface logic and buffering and increases wires and cells, while delay stays close to the baseline because cycle boundaries are preserved.

Overall. All metrics are reported as ratios to Yosys on the original RTL using the same synthesis and timing setup. On timing control flow and clock domain designs, LLM-aided methods do not show consistent improvements. Compared to these method, the compiler baseline changes less in these scenarios.

4.8 Answer to RQ3

Approach. We use our benchmark suite introduced above. For each original RTL in each of the four domains, we generate semantically equivalent mutants. We stratify code size by lines of code (LOC) at 100, 200, ..., 1000 and sample 100 mutants per size in each domain. For every optimization method, we apply the same procedure to the original RTL and to each mutant. We then run equivalence checking with ABC between the original RTL and its optimized output. If equivalence does not hold, we label the case as an optimization failure because the

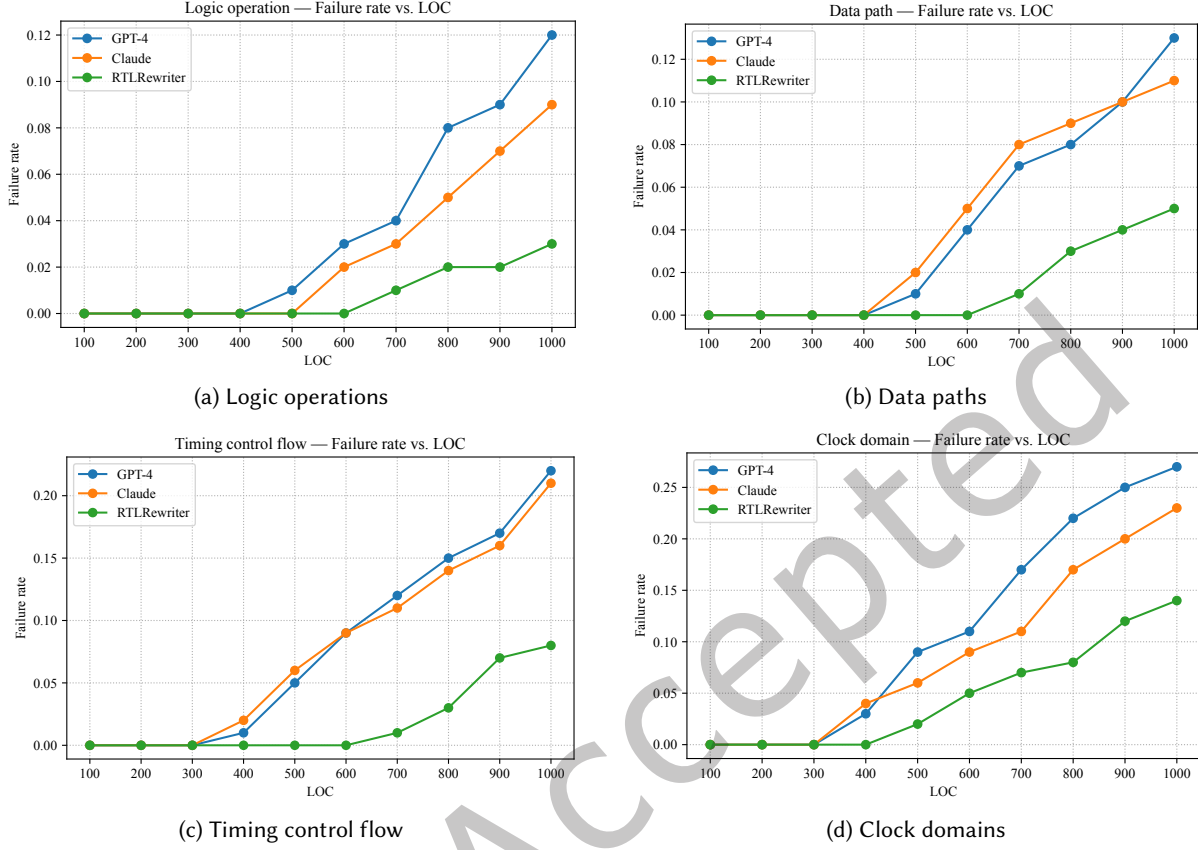


Fig. 6. Optimization failure rate (failures/100) versus lines of code (LOC, 100–1000) across four domains.

optimized RTL changes the functionality of the original. We report the failures rate for each method and for each LOC stratum in each domain.

Results. We report failure rate as failures divided by the number of evaluated cases at each lines-of-code (LOC) stratum. As shown in Figure 6, across domains, failure rates grow with LOC. For *logic operation*, failures remain near zero up to 400 LOC and rise at larger sizes: at 1000 LOC, GPT reaches 12%, Claude reaches 9%, and RTLRewriter reaches 3%. For *data path*, the trend is similar but slightly higher at large sizes: at 1000 LOC, GPT reaches 13%, Claude reaches 11%, and RTLRewriter reaches 5%. For *timing control flow*, failures appear earlier and increase faster: at 1000 LOC, GPT reaches 22%, Claude reaches 21%, and RTLRewriter reaches 8%. For *clock domain*, the increase is the steepest: at 1000 LOC, GPT reaches 27%, Claude reaches 23%, and RTLRewriter reaches 14%. Overall, logic operation is the most reliable, followed by data path, then timing control flow, and clock domain is the most error prone. The size sensitivity suggests that longer code fragments make it harder for models to preserve functional and temporal semantics during rewriting. The domain gap aligns with our earlier findings, when timing or cross-domain semantics are involved, incorrect rewrites are more likely. RTLRewriter yields the lowest failure rates in all domains and sizes. A plausible reason is its partition-and-rewrite strategy which limits the optimization context to smaller segments, which reduces cross-block inconsistencies that can

Table 4. RQ4: Optimization effectiveness under different prompts on equivalence-passing cases.

Method	Prompt	Logic Operation					Data Path					Timing Control Flow					Clock Domain				
		Wires	Cells	Delay	Area	Power	Wires	Cells	Delay	Area	Power	Wires	Cells	Delay	Area	Power	Wires	Cells	Delay	Area	Power
GPT_4	ZS	1.00	1.00	1.00	1.00	1.00	0.84↓	1.00	1.00	0.98↓	0.98↓	1.93↑	1.75↑	0.91↓	1.78↑	1.00	1.67↑	0.17↓	1.00	0.98↓	1.00
	FS	0.98↓	0.99↓	0.99↓	0.99↓	0.99↓	0.83↓	1.00	0.99↓	0.97↓	0.97↓	1.88↑	1.72↑	0.90↓	1.70↑	1.00	1.64↑	0.17↓	0.97↓	0.94↓	1.00
	CoT	0.97↓	0.99↓	0.99↓	0.98↓	0.98↓	0.82↓	1.00	0.99↓	0.96↓	0.96↓	1.86↑	1.70↑	0.89↓	1.65↑	1.00	1.62↑	0.16↓	0.96↓	0.94↓	1.00
Claude_3.7_Sonnet	ZS	1.00	1.00	1.00	1.00	0.99↓	0.84↓	1.00	1.00	0.98↓	0.98↓	1.00	1.00	1.00	1.00	1.00	1.67↑	0.22↓	0.96↓	0.96↓	1.00
	FS	0.99↓	0.99↓	0.99↓	0.99↓	0.99↓	0.83↓	1.00	0.99↓	0.97↓	0.97↓	0.99↓	0.99↓	1.00	0.98↓	1.00	1.64↑	0.22↓	0.90↓	0.91↓	1.00
	CoT	0.98↓	0.99↓	0.99↓	0.98↓	0.98↓	0.82↓	1.00	0.99↓	0.96↓	0.96↓	0.98↓	0.98↓	0.99↓	0.99↓	1.00	1.62↑	0.21↓	0.91↓	0.91↓	1.00
RTLRewriter	ZS	1.00	1.00	1.00	1.00	0.99↓	0.84↓	1.00	1.00	0.98↓	0.97↓	0.59↓	0.38↓	1.00	0.61↓	1.00	1.58↑	0.11↓	1.01↑	1.50↑	0.62↓
	FS	1.00	1.00	1.00	1.00	0.99↓	0.84↓	1.00	1.00	0.98↓	0.97↓	0.59↓	0.38↓	1.00	0.61↓	1.00	1.58↑	0.11↓	1.01↑	1.50↑	0.62↓
	CoT	1.00	1.00	1.00	1.00	0.99↓	0.84↓	1.00	1.00	0.98↓	0.97↓	0.59↓	0.38↓	1.00	0.61↓	1.00	1.58↑	0.11↓	1.01↑	1.50↑	0.62↓
Yosys (Ref.)	N/A	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

break equivalence. By contrast, directly optimizing long code requires the model to maintain wider dependencies; this correlates with higher failure rates at larger LOC.

Overall. Correctness degrades with code size and is most sensitive in designs that involve timing control and clock domains. RTLRewriter shows the lowest error rates, likely because partitioned rewriting constrains the edit scope and reduces context loss. GPT-4 and Claude-3.7-Sonnet remain reliable at small sizes but show higher failure rates as LOC increases, especially in timing control flow and clock domain cases.

4.9 Answer to RQ4

Approach. We study whether prompt engineering improves optimization quality. We consider zero-shot (ZS), Few-Shot (FS), Chain-of-Thought (CoT), whether prompt engineering improves optimization effectiveness. FS provides domain-specific examples that encode safe optimization patterns. CoT instructs the model to reason step by step before producing the optimized RTL and to return only the final code. Experiments are run on GPT-4 and Claude, and on RTLRewriter by only replacing its internal prompt while keeping its partitioning and Retrieval-Augmented Generation (RAG) unchanged. For each domain, we report absolute ratios for wires, cells, area, delay, and power, all normalized to Yosys on the original RTL.

Results. On logic operations and data paths, FS and CoT yield small but consistent improvements over ZS across wires and area, with delays close to baseline and minor power reductions. The FS examples anchor canonical Boolean refactoring templates, and step-by-step reasoning discourages unnecessary materialization of intermediate terms. As a result, conditional selection logic (MUX chains) become slightly shallower, which aligns with small area and power reductions without altering cycle boundaries. On timing control flow and clock domain designs, CoT and FS slightly reduce the regressions observed under ZS but the ratios remain above one in most metrics, indicating limited impact on temporally sensitive structures. Interface logic, widened synchronizer fan-out, and buffering remain the dominant contributors to area and wire increases. But when the prompt asks for a concrete timing rewrite, such as merging clock domains or simplifying cross domain interfaces, the models can improve the optimization effectiveness with small drops in delay and area. RTLRewriter shows no material difference between ZS, FS, and CoT in all four domains. Its retrieval and segmentation pipeline already constrains optimization to smaller code segments and supplies targeted exemplars, which limits additional effect from FS examples or step-by-step reasoning.

Overall. Prompt engineering can improve optimization effectiveness when local combinational refactoring dominates, namely logic operations and data paths. But its impact is limited on temporally governed optimizations in timing control flow and clock domain designs. Retrieval-augmented segmentation, as in RTLRewriter, already embeds example guidance and context control; FS and CoT therefore yield negligible additional benefit in this setting.

5 Discussion

5.1 Limitations of LLM-aided RTL Code Optimization Methods

Although LLM-aided RTL code optimization methods had shown promising results in optimizing RTL code, they still face several limitations. The primary limitation comes from the insufficient understanding of the semantics of the timing logic in RTL code. RTL, as a hardware description language, exhibits inherent concurrency, precise timing, and strict dependence on clock cycles, which significantly differ from the sequential text and software code-based training data and processing mechanisms of LLMs [67]. Furthermore, existing studies have demonstrated the general limitations of LLMs in broader temporal reasoning capabilities. For example, LLMs struggle with Allen interval relations, causal relationships, and long-term dependencies [26]. The inherent difficulty in interpreting clock-cycle semantics amplifies the typical black-box problem associated with LLMs. Consequently, this can lead to unreliable and non-explainable results during the process of assisting RTL code optimization [64].

Conventional synthesis tools such as ABC, Yosys, and commercial synthesizers remain the dominant solution for correctness-preserving logic optimization. Their passes operate on logic-level or technology-aware intermediate representations and are substantially more mature and reliable for timing, area, and power optimization. By contrast, the potential role of LLMs in our setting lies at the RTL source-code level: they can perform open-ended structural rewriting before synthesis, such as reorganizing expressions, decomposing transitions, and rewriting control logic beyond a fixed set of predefined passes. Our results show that this flexibility currently provides only limited benefit, mainly in simpler logic-operation and datapath cases, while timing-control-flow and clock-domain designs remain highly challenging.

However, these limitations do not imply that the problem is fundamentally intractable. Helping LLMs better understand the timing information in RTL code could be one of the solutions. For instance, in clock domain optimization, if the prompt includes detailed timing logic of the RTL and explicitly suggests that the LLM merge clock domains, the LLM could achieve better results. Therefore, future work could explore the use of temporally-aware neural networks, such as Spiking Neural Network [3, 20], as a front-end encoder for LLMs to help LLMs better understand the timing logic of RTL code. Beyond architectural improvements, several practical approaches could enhance LLM-aided RTL optimization. Multi-modal input integration could provide LLMs with explicit timing context by incorporating static timing analysis results, critical path information, and timing constraints as additional input modalities alongside the RTL code. This would help bridge the gap between structural code patterns and timing semantics. Domain-specific fine-tuning presents another promising direction. Training LLMs on curated datasets that emphasize timing-critical optimization patterns, FSM transformations, and clock domain crossing scenarios could improve their understanding of sequential behavior. Such datasets should include examples of both successful timing optimizations and common timing pitfalls to help models learn timing-aware decision making. Furthermore, incorporating formal verification and timing analysis into the LLM optimization loop could ensure correctness and timing compliance. Real-time feedback from timing analyzers could guide LLM suggestions toward timing-feasible solutions, while formal equivalence checking could prevent functional errors during optimization iterations.

5.2 Threats

This paper has two main threats to validity. First, the threats in benchmark construction. The quality of the RTL code could affect LLM-aided RTL code optimization. To alleviate this threat, we carefully checked the RTL code in our Benchmark and adjusted the RTL code which is not representative of the real-world RTL code. Second, the generalizability to other LLMs. In this study, we only studied two LLMs and one open-source LLM-aided method due to time and hardware limits. However, there are many other LLMs that could be used for RTL code optimization. To alleviate this threat, we selected the SOTA LLMs in different families as representatives. Furthermore, there are some LLM-aided methods that can be used for RTL code optimization, such as VeriGen [56] and RTLCoder [36]. But due to the fact that they mainly focus on RTL code generation rather than optimization, we did not include them in our experiments.

6 Related Work

6.1 Code Optimization

Code optimization serves as a foundation for various software engineering (SE) tasks [37]. Code optimization involves transforming programs at various levels, such as source code [53], compiler intermediate representation [15, 68], or binary code [4, 17, 33, 42] to achieve specific performance goals, such as reducing execution time [40], minimizing code size [22], or optimizing memory usage [19]. One key challenge in code optimization is the enormous search space, where effective solutions are rare and exhaustive search is computationally impractical [47]. Traditionally, code optimization has relied on expert-designed heuristics and rules [31, 50]. Over time, a diverse array of optimization techniques has emerged, ranging from low-level strategies, such as instruction scheduling [18], register allocation [8], vectorization [11], and loop transformations [61] to high-level strategies that modify algorithms or data structures at the source code level to improve performance [63]. Over the past decades, a substantial body of work has explored the use of machine learning for code optimization. For example, Liang et al. used deep reinforcement learning to discover pass sequences that reduce both runtime and code size, outperforming standard compiler pipelines for certain benchmarks [32]. Cummins et al. proposed ProGraML, it learns graph embeddings of LLVM IR and has been applied to predict optimal optimization passes or to classify programs by optimization difficulty [14]. With the gradual maturity of large language models (LLMs), their application in code optimization has begun to gain attention. Although LLM-aided code optimization is still at an early stage, some results show that LLMs can occasionally suggest high-level changes, such as adopting alternative algorithms or libraries [21]. Nevertheless, LLM-driven code optimization remains immature and faces substantial limitations. Therefore, systematic evaluation methods are needed to assess the effectiveness of LLM-aided code optimization methods.

6.2 RTL Code Optimization

Register Transfer Level (RTL) code optimization is crucial for improving the efficiency and performance of digital circuits during the early synthesis stages. In the domain of RTL code optimization, the exploration spans several critical dimensions. For logic operation optimization, researchers focus on subexpression elimination [12, 45], constant folding [12], constant propagation [41, 66] and algebraic simplification [5, 7]. Concerning datapath optimization, techniques mainly include dead code elimination [23, 28], strength reduction [13] Mux reduction [10, 65], Mux tree decomposition [48], and Mux tree restructuring [65]. When focusing on memory optimization, the emphasis is memory sharing [29, 39], memory folding, memory banking [30, 73], and memory pipelining [44]. Finally, within finite state machine (FSM) design optimizations, the primary focus is on state minimization [24], state assignment [62], and state decomposition [52]. Although these techniques are effective, the lack of a centralized repository means that optimization patterns in design manuals and code libraries remain underutilized. This forces engineers to rely more on their expertise rather than on these automated tools.

Recent studies have demonstrated the potential of leveraging the powerful generative capabilities of LLMs to automatically rewrite and optimize RTL code [36, 55, 72]. For example, Yao et al. proposed RTLRewriter, a method that uses LLMs to rewrite RTL code based on abstract syntax trees (ASTs) and domain-specific knowledge [72]. It can effectively optimize logic operations, data paths, and timing control flow in RTL code. Recent work has also explored PPA-aware Verilog generation with LLMs. In particular, LLM-VeriPPA combines Verilog generation, simulator-guided repair, and synthesis-feedback-based PPA-aware refinement. This is closely related to our study, but its primary focus is on Verilog code generation and refinement from textual specifications. These methods still face several limitations, including the inability to align their outputs with specified optimization objectives, inefficiencies in the synthesis process, and a lack of timing-logic optimization. Therefore, a comprehensive evaluation method is needed to systematically assess the effectiveness of LLM-aided RTL code rewriting and optimization methods.

7 Conclusion and Future Work

We presented a metamorphic testing-based evaluation and a companion benchmark for LLM-aided RTL optimization. The suite covers four domains common in practice: logic operations, data paths, timing control flow, and clock domains. For logic operations and data paths, LLM-aided methods generally reduce wire count and area while keeping delay near the baseline. These effects hold in equivalent forms of the code. For timing control flow and clock domains, gains are limited. Structural measures often increase with transformed code and area, while delay remains near the baseline. Failure rates increase with code size, especially for timing and cross-domain designs. Segmentation reduces failures but does not change these trends. Furthermore, prompt engineering offers small, consistent gains in logic operations and data paths. Few-shot examples and chain-of-thought planning help models avoid unnecessary materialization of intermediate terms. Their impact on timing control flow and clock domains is modest because they do not change cycle boundaries or synchronizer placement. When the instruction includes explicit timing intent, small gains appear in some cases. We see two directions for future work. The first is timing-aware optimization, which uses global state transition graphs and clock-domain analysis to guide rewriting. The second is stronger correctness feedback-in-the-loop, including equivalence checking and constraint solving during code generation. We release our benchmark and scripts to support repeatable evaluation.

8 Declaration of Generative AI Use.

During the preparation of this manuscript, the authors used generative AI tools for limited language polishing and used Gemini to assist with an early draft of Fig. 5. The final figure was manually redrawn and verified by the authors.

References

- [1] 2023. ICARUS Verilog. <https://github.com/steveicarus/iverilog>.
- [2] AMD (Xilinx). 2023. *Vivado Design Suite User Guide: Synthesis and Timing Closure (UG949)*. Accessed: 2025-07-22.
- [3] Malyaban Bal and Abhronil Sengupta. 2024. Spikingbert: Distilling bert to train spiking language models using implicit differentiation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 10998–11006.
- [4] M Ammar Ben Khadra, Dominik Stoffel, and Wolfgang Kunz. 2020. Efficient binary-level coverage analysis. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1153–1164.
- [5] Bruno Buchberger and Rüdiger Loos. 1982. Algebraic simplification. In *Computer algebra: symbolic and algebraic computation*. Springer, 11–43.
- [6] Cadence Design Systems. 2014. Clock-domain and reset-domain crossing in low-power design. <https://www.techdesignforums.com/practice/technique/clock-domain-and-reset-verification-low-power/>. *Tech Design Forum* (2014). Accessed: 2025-07-22.
- [7] Jacques Carette. 2004. Understanding expression simplification. In *Proceedings of the 2004 international symposium on Symbolic and algebraic computation*. 72–79.

- [8] Gregory J Chaitin. 1982. Register allocation & spilling via graph coloring. *ACM Sigplan Notices* 17, 6 (1982), 98–101.
- [9] Deming Chen. 2009. , Design Automation for Microelectronics, Springer Handbook of Automation. *icims. cs. uiuc. edu* (2009).
- [10] Deming Chen and Jason Cong. 2004. Register binding and port assignment for multiplexer optimization. In *ASP-DAC 2004: Asia and South Pacific Design Automation Conference 2004 (IEEE Cat. No. 04EX753)*. IEEE, 68–73.
- [11] Yishen Chen, Charith Mendis, and Saman Amarasinghe. 2022. All you need is superword-level parallelism: systematic control-flow vectorization with SLP. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 301–315.
- [12] John Cocke. 1970. Global common subexpression elimination. In *Proceedings of a symposium on Compiler optimization*. 20–24.
- [13] Keith D Cooper, L Taylor Simpson, and Christopher A Vick. 2001. Operator strength reduction. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 23, 5 (2001), 603–625.
- [14] Chris Cummins, Zacharias V Fisches, Tal Ben-Nun, Torsten Hoefer, Michael FP O’Boyle, and Hugh Leather. 2021. Programl: A graph-based program representation for data flow analysis and compiler optimizations. In *International Conference on Machine Learning*. PMLR, 2244–2253.
- [15] Chris Cummins, Volker Seeker, Dejan Grubisic, Baptiste Roziere, Jonas Gehring, Gabriel Synnaeve, and Hugh Leather. 2024. Meta large language model compiler: Foundation models of compiler optimization. *arXiv preprint arXiv:2407.02524* (2024).
- [16] Doulos Ltd. 2020. FSM Optimization: KnowHow Tutorial. <https://www.doulos.com/knowhow/fpga/fsm-optimization/>. Accessed: 2025-07-22.
- [17] Mary F Fernandez. 1995. Simple and effective link-time optimization of Modula-3 programs. In *Proceedings of the ACM SIGPLAN 1995 conference on Programming language design and implementation*. 103–115.
- [18] Olivier Flückiger, Guido Chari, Ming-Ho Yee, Jan Ječmen, Jakob Hain, and Jan Vitek. 2020. Contextual dispatch for function specialization. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–24.
- [19] Spandan Garg, Roshanak Zilouchian Moghaddam, Colin B Clement, Neel Sundaresan, and Chen Wu. 2022. DeepDev-PERF: a deep learning-based approach for improving software performance. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 948–958.
- [20] Samanwoy Ghosh-Dastidar and Hojjat Adeli. 2009. Spiking neural networks. *International journal of neural systems* 19, 04 (2009), 295–308.
- [21] Jingzhi Gong, Vardan Voskanyan, Paul Brookes, Fan Wu, Wei Jie, Jie Xu, Rafail Giavrimis, Mike Basios, Leslie Kanthan, and Zheng Wang. 2025. Language models for code optimization: Survey, challenges and future directions. *arXiv preprint arXiv:2501.01277* (2025).
- [22] Dejan Grubisic, Volker Seeker, Gabriel Synnaeve, Hugh Leather, John Mellor-Crummey, and Chris Cummins. 2024. Priority sampling of large language models for compilers. In *Proceedings of the 4th Workshop on Machine Learning and Systems*. 91–97.
- [23] Rajiv Gupta, DA Benson, and Jesse Zhixi Fang. 1997. Path profile guided partial dead code elimination using predication. In *Proceedings 1997 International Conference on Parallel Architectures and Compilation Techniques*. IEEE, 102–113.
- [24] MS Harris. 1998. Synthesis of finite state machines: Functional optimization. *Microelectronics Journal* 29, 6 (1998), 364–365.
- [25] Jordan Henkel, Goutham Ramakrishnan, Zi Wang, Aws Albarghouti, Somesh Jha, and Thomas Reps. 2022. Semantic robustness of models of source code. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 526–537.
- [26] Duygu Sezen Islakoglu and Jan-Christoph Kalo. 2025. ChronoSense: Exploring Temporal Understanding in Large Language Models with Time Intervals of Events. *arXiv preprint arXiv:2501.03040* (2025).
- [27] Xin Jin, Kexin Pei, Jun Yeon Won, and Zhiqiang Lin. 2022. Symlm: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1631–1645.
- [28] Jens Knoop, Oliver Rüthing, and Bernhard Steffen. 1994. Partial dead code elimination. *ACM Sigplan Notices* 29, 6 (1994), 147–158.
- [29] Charles Eric LaForest and J Gregory Steffan. 2010. Efficient multi-ported memories for FPGAs. In *Proceedings of the 18th annual ACM/SIGDA international symposium on Field programmable gate arrays*. 41–50.
- [30] Bo-Cheng Lai, Bo-Ya Chen, Bo-En Chen, and Yi-Da Hsin. 2019. REMAP+: An efficient banking architecture for multiple writes of algorithmic memory. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 28, 3 (2019), 660–671.
- [31] Andrea Lepori, Alexandru Calotoiu, and Torsten Hoefer. 2025. Iterating Pointers: Enabling Static Analysis for Loop-based Pointers. *ACM Transactions on Architecture and Code Optimization* 22, 1 (2025), 1–25.
- [32] Youwei Liang, Kevin Stone, Ali Shamel, Chris Cummins, Mostafa Elhoushi, Jiadong Guo, Benoit Steiner, Xiaomeng Yang, Pengtao Xie, Hugh James Leather, et al. 2023. Learning compiler pass orders using coreset and normalized value prediction. In *International Conference on Machine Learning*. PMLR, 20746–20762.
- [33] Nandor Licker and Timothy M Jones. 2020. Duplo: a framework for OCaml post-link optimisation. *Proceedings of the ACM on Programming Languages* 4, ICFP (2020), 1–29.
- [34] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating large language models for verilog code generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–8.

- [35] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating large language models for verilog code generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–8.
- [36] Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2024. RTLCode: Fully open-source and efficient LLM-assisted RTL code generation technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024).
- [37] Edward S Lowry and Cleburne W Medlock. 1969. Object code optimization. *Commun. ACM* 12, 1 (1969), 13–22.
- [38] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. Rtlm: An open-source benchmark for design rtl generation with large language model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 722–727.
- [39] Jiacheng Ma, Gefei Zuo, Kevin Loughlin, Xiaohe Cheng, Yanqiang Liu, Abel Mulugeta Eneyew, Zhengwei Qi, and Baris Kasikci. 2020. A hypervisor for shared-memory FPGA platforms. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 827–844.
- [40] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* 36 (2023), 46534–46594.
- [41] Robert Metzger and Sean Stroud. 1993. Interprocedural constant propagation: An empirical study. *ACM Letters on Programming Languages and Systems (LOPLAS)* 2, 1-4 (1993), 213–232.
- [42] Angélica Aparecida Moreira, Guilherme Ottoni, and Fernando Magno Quintão Pereira. 2021. Vespa: static profiling for binary optimization. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (2021), 1–28.
- [43] OpenCores. [n.d.]. OpenCores. <https://opencores.org/>. Accessed: 2026-03-28.
- [44] Joonseok Park and Pedro C Diniz. 2001. Synthesis of pipelined memory access controllers for streamed data applications on FPGA-based computing engines. In *Proceedings of the 14th international symposium on Systems synthesis*. 221–226.
- [45] Robert Pasko, Patrick Schaumont, Veerle Derudder, Serge Vernalde, and Daniela Durackova. 1999. A new algorithm for elimination of common subexpressions. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 18, 1 (1999), 58–68.
- [46] Kexin Pei, Weichen Li, Qirui Jin, Shuyang Liu, Scott Geng, Lorenzo Cavallaro, Junfeng Yang, and Suman Jana. 2024. Exploiting Code Symmetries for Learning Program Semantics. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*.
- [47] Phitchaya Mangpo Phothilimthana, Aditya Thakur, Rastislav Bodik, and Dinakar Dhurjati. 2016. Scaling up superoptimization. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*. 297–310.
- [48] Peter Pišteka, Katarína Jelemenská, and Milan Kolesár. [n.d.]. Reduction of Multiplexer Trees using Modified Lookup Table. ([n.d.]).
- [49] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [50] Anjan Kumar Sarma. 2015. New trends and challenges in source code optimization. *International Journal of Computer Applications* 131, 16 (2015), 27–32.
- [51] Tsutomu Sasao. 1993. *Logic synthesis and optimization*. Vol. 2. Springer.
- [52] R.S. Shelar, M.P. Desai, and H. Narayanan. 1999. Decomposition of finite state machines for area, delay minimization. In *Proceedings 1999 IEEE International Conference on Computer Design: VLSI in Computers and Processors (Cat. No.99CB37040)*. 620–625.
- [53] Alexander Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob Gardner, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. 2023. Learning performance-improving code edits. *arXiv preprint arXiv:2302.07867* (2023).
- [54] Synopsys Inc. 2011. *Design Compiler Optimization Reference Manual* (vf-2011.09 ed.). Synopsys. <https://picture.iczhiku.com/resource/eetop/SHidRGQWtQruovNN.pdf>.
- [55] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. Verigen: A large language model for verilog code generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024), 1–31.
- [56] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. Verigen: A large language model for verilog code generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024), 1–31.
- [57] YunDa Tsai, Mingjie Liu, and Haoxing Ren. 2024. RTLFixer: Automatically fixing RTL syntax errors with large language model. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [58] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini. 2023. Can large language models identify and reason about security vulnerabilities? not yet. *arXiv preprint arXiv:2312.12575* (2023).
- [59] Supriya Unnikrishnan and Sujith M. Iype. 2024. *Reducing Power Hot Spots through RTL Optimization Techniques*. Accessed: 2025-07-22.
- [60] Vemeko Blog. 2025. What techniques can be used to reduce power consumption in FPGAs? <https://www.vemeko.com/blog/67173.html>. Accessed: 2025-07-22.

- [61] Anand Venkat, Mary Hall, and Michelle Strout. 2015. Loop and data transformations for sparse matrix code. *ACM SIGPLAN Notices* 50, 6 (2015), 521–532.
- [62] Tiziano Villa, Timothy Kam, Robert K Brayton, and Alberto L Sangiovanni-Vincentelli. 2012. *Synthesis of finite state machines: logic optimization*. Springer Science & Business Media.
- [63] Michael M Vitousek, Jeremy G Siek, and Avik Chaudhuri. 2019. Optimizing and evaluating transient gradual typing. In *Proceedings of the 15th ACM SIGPLAN international symposium on dynamic languages*. 28–41.
- [64] Yiting Wang, Wanghao Ye, Ping Guo, Yexiao He, Ziyao Wang, Bowei Tian, Shwai He, Guoheng Sun, Zheyu Shen, Sihan Chen, et al. 2025. SymRTLO: Enhancing RTL Code Optimization with LLMs and Neuron-Inspired Symbolic Reasoning. *arXiv preprint arXiv:2504.10369* (2025).
- [65] Zicheng Wang, Hailong You, Jie Wang, Meihua Liu, Yu Su, and Yong Zhang. 2023. Optimization of multiplexer combination in rtl logic synthesis. In *2023 International Symposium of Electronics Design Automation (ISED)*. IEEE, 121–125.
- [66] Mark N Wegman and F Kenneth Zadeck. 1991. Constant propagation with conditional branches. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 13, 2 (1991), 181–210.
- [67] Anjiang Wei, Huanmi Tan, Tarun Suresh, Daniel Mendoza, Thiago SFX Teixeira, Ke Wang, Caroline Trippel, and Alex Aiken. 2025. VeriCoder: Enhancing LLM-Based RTL Code Generation through Functional Correctness Validation. *arXiv preprint arXiv:2504.15659* (2025).
- [68] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. Egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages* 5, POPL (2021), 1–29.
- [69] Clifford Wolf, Johann Glaser, and Johannes Kepler. 2013. Yosys-a free Verilog synthesis suite. In *Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip)*, Vol. 97.
- [70] Zhihao Xu, Shikai Guo, Xiaochen Li, Zun Wang, and He Jiang. 2025. SIMTAM: Generation Diversity Test Programs for FPGA Simulation Tools Testing Via Timing Area Mutation. *ACM Transactions on Design Automation of Electronic Systems* 30, 2 (2025), 1–25.
- [71] Zhihao Xu, Shikai Guo, Guilin Zhao, Peiyu Zou, Xiaochen Li, and He Jiang. 2025. A novel HDL code generator for effectively testing FPGA logic synthesis compilers. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).
- [72] Xufeng Yao, Yiwen Wang, Xing Li, Yingzhao Lian, Ran Chen, Lei Chen, Mingxuan Yuan, Hong Xu, and Bei Yu. 2024. Rtlrewriter: Methodologies for large models aided rtl code optimization. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–7.
- [73] Yuan Zhou, Khalid Musa Al-Hawaj, and Zhiru Zhang. 2017. A new approach to automatic memory banking using trace-based address mining. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 179–188.
- [74] Reto Zimmermann and Arshid Syed. 2005. RTL Coding Guidelines for Datapath Synthesis. Presentation, Synopsys Users Group (SNUG) Boston. <https://picture.iczhiku.com/resource/eetop/wyiEELySpYPudnxV.pdf>.